Valiant Verification SDK

A technical explainer — what "verified" honestly means, and the machine that makes the measurement real.

---

0 · What it guarantees (read this first)

The SDK does **not** know how to "make code verified", and it does **not** guarantee "working code". Guaranteeing that arbitrary software is correct is undecidable — it is the halting problem. The SDK is a *meta*-tool: **it verifies the verification, not the code.**

It does three concrete things, and nothing else:

MOVE	WHAT IT DOES
1 · run your checks honestly	You bring the checks (tests, typecheck, lint, build, your rules). It runs them and returns the truth: `0` holds, `1` broken, `2` could not run. A skipped or silent check is a *fail*, never a pass.
2 · prove the checks can catch a bug	It breaks your code on purpose (deletes a function, inverts a condition) and requires the check to go red. A check that stays green while its subject is broken is `SURVIVED` — refused.
3 · enforce that it happened	Hooks block the agent/CI at completion when the gate is red. There is an exit code and a record, not a claim.

IT *DOES* GUARANTEE	IT DOES *NOT* GUARANTEE
the checks ran (not skipped)	the code is correct for all inputs
each check was shown to fail on a real breakage	you wrote *enough* checks — coverage is yours
the result is a number, not an adjective	you checked the *right* things
the record is tamper-evident	

> **One-sentence version:** it is a calibrated scale, not a chef. A scale doesn't make the food good — it reports the weight honestly and you can test the scale with a known weight. VVS is the scale for "verified": it makes the measurement honest and provable, and it proves the scale isn't lying. How much of your code the checks touch is still your job.

---

1 · What it is

The Valiant Verification SDK is a **contract-checking layer that ships with a proof that each check can actually fail**. It runs your project's own checks, deliberately breaks the thing each check protects, and requires the check to go red on the breakage. The result is an exit-code contract — `0` holds, `1` broken, `2` could not be checked — enforced at the agent, in CI, and on a human control panel. A green result means "a machine tried to break your rules and they held", never "the tests passed".

---

2 · The verification loop

2.1 The gate — a check that must return a number

Every check is a named contract that returns an exit code. `2` is the important one: most CI treats "no output / skipped" as a pass, but VVS treats it as a *fail* — a check that didn't run, or a gate that printed nothing and exited 0, is a defect, not a green.

CODE	MEANING
`0`	the contract holds
`1`	the contract is broken
`2`	the contract could not be checked — never a pass

2.2 Eight failure modes, each named

A red gate always says *which* failure mode, never just "failed":

VERDICT	MEANING	WHAT TO DO
CAUGHT	the check went red when its subject was broken, green when restored	the guard is real
SURVIVED	the check *passed* with the subject broken	the check doesn't test what you think
STALE	the sabotage's `find` matched zero or multiple sites	fix the string — a no-op sabotage proves nothing
UNSTABLE	the check stayed red after the breakage was undone	state outside the copy is leaking in
ALWAYS-RED	the check fails with nothing sabotaged	the check is broken, not the guard
CANNOT-RUN	the check could not run, or answered `2`	fix the environment — not a detection

---

3 · Mutation testing — prove the check can fail

Verifying "the check passes" is half the story. VVS also requires **"the check goes red when the thing it protects is broken"**. `valiant mutate` sabotages the subject — deletes a file, flips a flag, inverts a condition — then requires the guard to catch it.

A guard that has never been shown to fail is refused. That is the difference between "we have 100 tests" and "we have 100 tests that can detect a bug". The sabotage is deterministic (a single `find` / `replace` site), so the mutation is reproducible, and the verdict is evidence, not an opinion.

> **A 20-second concrete example.** A repo has one check: `npm test`, with a test asserting `sum(2,3) === 5`. `valiant mutate` rewrites `return a + b` to `return a - b`. The check must now go red — and it does (**CAUGHT**). Had it stayed green, the SDK would report **SURVIVED** and refuse to call the check real. That is the whole mechanism: it doesn't know what `sum` should do — *you* told it via the test — it only proves your test would notice if that broke.

---

4 · Self-verification — the SDK verifies itself first

`SELF.json` is a sha256 manifest of every shipped file (`bin/`, `src/`, `corpus/`, `tools/`, `valiant.json`). Before the SDK verifies *your* repo it verifies *itself*: if any SDK file changed, the `selfcheck` gate fails. A tampered or partially-updated

SDK cannot silently produce a green.

---

5 · The corpus — rules that ship

The SDK ships a corpus of battle-tested contracts. Each invariant traces to a real incident; the publisher refuses an entry with no real failure behind it.

COMPONENT	COUNT	WHAT IT IS
Universal invariants	46	the rules every claim must hold
Hazards that bought them	56	the real failures each invariant prevents (52 universal, 4 product-specific and not shipped)
Operating-method rules	24	the discipline: memory, verify, build order, reporting
Reference guards	29	the code that enforces the invariants, run on `verify --corpus`

5.1 The 46 invariants

1 Contract integrity · **2** Silent no-op edits are forbidden · **3** Bound the model's output · **4** Schema ships with its migration · **5** Sync against the live source of truth · **6** Verify before depending · **7** Schema drift degrades gracefully · **8** A module is not verified until it is loaded · **9** A silent zero is a bug · **10** Never neutralise a build step · **11** Review behaviour, not just the export surface · **12** Data isolation is a decision · **13** "Nothing failed" is not "the gates ran" · **14** A declaration outlives its block · **15** Prove the check can fail · **16** Name the machines the output runs on · **17** The verdict is the build's · **18** Lint coverage, render the real surface · **19** A tool that rewrites the tree owns it · **20** Close stdin on spawned subprocesses · **21** Portable filenames · **22** The loop survives its own queue · **23** A refresh preserves what it cannot derive · **24** A tool that cannot do its job exits non-zero · **25** Liveness is proved, not inferred · **26** A repeated action cannot run faster than its interval · **27** Evidence is recomputed, never transcribed · **28** A stalled process is distinguishable from a slow one · **29** A pipeline must not mask the status it exists to report · **30** A generated file is edited at its generator · **31** A parser is not a type-checker · **32** Never edit what you have not read · **33** Never edit what you do not understand · **34** A claim of inability is checked, not announced · **35** A measurement under load measures the rig · **36** Stable context first, volatile last · **37** Search before building · **38** Absence of a capability is a task · **39** Blast radius before action · **40** A UI change is verified by looking at it · **41** No dead ends · **42** Evidence reaches the verifier · **43** A wait loop attempts the repair it waits for · **44** AI action approved · **45** AI claim traceable · **46** AI intent restated.

5.2 The 24 method rules

M1 Write it down or it is lost · **M2** The injected file must stay small · **M3** Record the incident with the rule · **M4** Handover is pull, not push · **M5** A stale handover is worse than none · **M6** Never claim, always measure · **M7** Prove the check can fail · **M8** Distinguish cannot-check from pass · **M9** Ask the gate, don't parse its source · **M10** A green local check says nothing about the other list · **M11** Smallest change that can be verified · **M12** Root-cause before patching · **M13** Fix the generator, not its output · **M14** After every incident, write the rule AND the guard · **M15** One workspace per worker, check before commit · **M16** A job that fails in seconds is a missing name, not a slow test · **M17** Say what is NOT done · **M18** Evidence travels with the claim · **M19** Never wait on a queue you could bypass · **M20** Do the next authorised step · **M21** Turn every reminder into a rule or a gate · **M22** Probe before declaring a wall · **M23** Check the blast radius before acting · **M24** Pre-authorise the reversible.

5.3 The 56 hazards (each invariant traces to a real incident)

A shared module loses an export / changes a return shape. An admin query over-filters by `created_by_id` and hides system rows. A generated file edited by hand, lost on rebuild. A lockfile not committed, so an install rewrites it. An edit whose `find` matches zero or multiple sites ships nothing silently. An unbounded model call truncated by a provider default. A push wiping protected paths. A schema change without its migration. A push diffing against a stale local snapshot. Code written against an API shape from memory. A new column breaking a feature until the SQL lands. A wrong relative import passing gates, killing the container at boot. An external query returning empty forever with nothing saying so. A green build shipping an artifact that can't run. A shared module losing behaviour but keeping its export surface. An admin read exposing another account's private data. A check that never ran reading as passed. A `try` block hiding declarations from its catch. A guard satisfied by the very bug it exists to prevent. A build running only on its own machine, unnamed. A failed save reading as a failed build. A lint-blocked file checked by no rules. A tree-rewriter restoring a stale backup over live edits. A subprocess prompting on inherited stdin, blocking the loop. A filename Windows can't represent. A loop killable by its own queue. A refresh dropping hand-added fields. A repair tool reporting a sick node healthy with exit 0. A board reading a dead node green from a cached signal. A restart script restarting a service every 41 seconds. A badge counting hand-transcribed evidence. A wedged loop with no last-progress signal reading as "slow". A drift check silently removed from the loop. Operating without reading the rules the system carries. Instructions that didn't match the system they described. A stale record trusted over the live machine. A live-check claim made without a live check. Hedging on a finished delivery instead of stating the result. Work finished with no handoff. A rule ported between systems without validation. A job printing PASS while both checks failed, because it piped into a filter. A live config edited directly, overwritten by the installer. Six type errors reaching a gate because a formatter parsed the file. An edit applied to an unread file, anchor missing silently. A flake "found" on a machine that was simultaneously compiling. An instruction file grown until truncated, dropping the safety rules. Two sessions building the same thing, discovered by disagreement. A missing tool reported as a wall instead of installed. A local change taking down a shared capability, no blast-radius check. A UI change merged unlooked-at. An error state with no next step. A guard edited in the deployed copy, fix vanishing on sync. Code changed to hide a symptom in a function the author couldn't explain. An abandoned download rendered in-flight for five days. A flashed image refusing its SSH key, diagnosed by rebuilding five times. Sleep-polling a queued CI job ~30 times while five idle nodes sat there.

5.4 The 29 reference guards (code → invariant)

GUARD	INVARIANT
`contract-integrity`	1 — contract integrity
`resolve-imports`	8 — a module is verified when loaded
`model-call-budget`	3 — bound the model's output
`convention-queries-bounded`	9 + 7 — silent zero / schema drift
`no-swallowed-build`	10 — never neutralise a build step
`tenant-predicate`	12 — data isolation
`declaration-scope`	14 — a declaration outlives its block
`ci-can-skip`	17 — a skipped check isn't a pass
`lint-coverage`	18 — lint coverage
`closed-stdin`	20 — close stdin on subprocesses
`portable-filenames`	21 — portable filenames
`convention-loop-survives`	22 — the loop survives its queue
`convention-refresh-preserves`	23 — a refresh preserves
`nonzero-on-failure`	24 — exit non-zero on failure
`convention-status-proved`	25 — liveness proved
`timers-that-loop`	26 — no faster-than-interval repeats
`progress-visible`	28 — stalled vs slow
`pipeline-masks-status`	29 — no masked status
`generated-file`	30 — edit at the generator
`typecheck-is-real`	31 — a parser isn't a type-checker
`convention-measure-load`	35 — measure the rig, not the code
`convention-context-order`	36 — stable first, volatile last
`convention-ui-screenshot`	40 — UI verified by looking
`convention-errors-actionable`	41 — no dead ends
`evidence-reaches-verifier`	42 — evidence reaches the verifier
`wait-must-try-to-fix`	43 — a wait loop attempts the repair
`ai-control-approved`	44 — AI action approved
`ai-control-traceable`	45 — AI claim traceable
`ai-control-restated`	46 — AI intent restated

6 · Enforcement — not just a report, a block

The SDK has an **enforcement stance** — `enforced` or `advisory` — and it installs hooks into the agent and CI.

Hooks. A PreToolUse / Stop hook runs `valiant gate` at the moment an agent tries to *finish* (claim completion). A non-zero exit blocks the agent and hands it the reason. The hook only gates commands that would claim completion (e.g. `git commit/push`, `gh pr create/merge`) — gating every tool call in every directory is how a correct gate becomes an unusable one.

MCP. `valiant mcp` exposes five tools — `valiant_gate`, `valiant_verify`, `valiant_ledger`, `valiant_outcomes`, `valiant_licence_status` — so an agent can ask for the verdict as structured data rather than parsing stdout.

7 · The AI-control layer — human in the loop, as a number

This answers a different question than "does the code pass": **did the AI stay under human control?** Every action the agent takes is appended to `ai-actions.jsonl` (NDJSON) with three fields, and three guards check them:

GUARD	WHAT IT ENFORCES
`ai-control-approved`	every action carries a human "yes" (`approved: true`); pending or rejected is a control break
`ai-control-traceable`	every claim points at a source a human can re-open
`ai-control-restated`	the agent restated what it meant before it acted (the asked-vs-meant gap)

`valiant control` reduces this to one number — `0` under control · `1` a control broke · `2` could not check. The human ticks yes/no on each logged action (`--approve` / `--reject`, or the GUI's yes/no buttons), and `valiant control --prove` runs each guard's seeded violation so a third party can confirm the guards actually catch failures. The ledger is the evidence: every action has a human's name on it, so "rogue AI" can't be the end of the sentence.

8 · The licence — an offline, signed credential

The licence is an **ed25519-signed envelope** — `vvs1.<payload>.<signature>` — carrying `{ plan, seats, iat, exp, kid, jti }`. The SDK verifies it **offline** against a shipped public key (`licence-public.pem`): no call home, no licence server to depend on. Revocation is a signed list the SDK can pull (`--sync`). The private key never leaves the issuer's machine. Every plan gets the whole tool — a licence that unlocks a partial product teaches a customer the product is partial.

9 · The surfaces and the end-to-end flow

SURFACE	COMMAND	WHAT IT IS
CLI	`init / setup / gate / mutate / verify / control / mcp / ui`	the whole tool in one binary
Hooks	`enforce --install`	blocks the agent at completion
MCP	`valiant mcp`	five tools for the agent

GUI	`valiant ui`	a local web panel — toggle enforcement, watch the gate stream, tick the control ledger, activate a licence, run setup

```
curl -fsSL https://vvs.morpheus.nz/install.sh | sh   # install, opens the control panel
valiant setup                                        # valiant.json + a proposed, proven guard
valiant mutate                                       # break the subject → the guard must go red
valiant gate                                         # exit 0/1/2 → what CI + hooks enforce
valiant verify                                       # your gates + corpus guards + selfcheck
valiant control                                      # 0/1/2 — did the AI stay under human control
```

`valiant.json` is the single config file the SDK adds to a repo — it names the gates, the guards, and the enforcement stance. That is the whole surface area: one config, one exit-code contract, and a proof that every check can fail.

> **The mental model for an engineer.** VVS is to *verification claims* what a property-based test + mutation-testing harness + pre-commit hook are to a test suite, applied uniformly — and it refuses to call anything "verified" that it hasn't first shown can detect a real, deliberately-introduced failure.
